DZIEN 1 — PODSTAWY UIPATH STUDIO, ZMIENNE, EXCEL I DATATABLE

Modul 1 — Czym jest UiPath Studio?

UiPath Studio to srodowisko programistyczne (IDE) do tworzenia robotow RPA. Robot RPA nasladuje czynnosci uzytkownika — klika, wpisuje tekst, czyta dane z ekranu, przenosi pliki — ale robi to automatycznie, bez przerw i bledow wynikajacych ze zmeczenia.

Najwazniejsze koncepcje na start

- **Workflow** — przeplyw pracy, czyli sekwencja krokow, ktore robot wykonuje. Zapisywany jako plik `.xaml`.
- **Activity (Aktywnosc)** — pojedynczy krok w workflow (np. klikniecie przycisku, wpisanie tekstu, otwarcie przegladarki). UiPath ma ponad 500 wbudowanych aktywnosci.
- **Sequence** — kontener, w ktorym aktywnosci wykonuja sie liniowo, jedna po drugiej (z gory na dol). To najczesciej uzywany typ kontenera.
- **Project** — caly projekt automatyzacji, zawierajacy workflow, zaleznosci, pliki danych.

Interfejs UiPath Studio — kluczowe panele

Panel	Opis	Kiedy uzywac
Activities	Lista wszystkich dostepnych aktywnosci (szukaj po nazwie!)	Gdy szukasz konkretnej aktywnosci
Properties	Wlasciwosci zaznaczonej aktywnosci (Input, Output, opcje)	Zawsze po dodaniu aktywnosci
Variables	Lista zmiennych w biezacym scope	Zarzadzanie zmiennymi
Output	Logi z wykonania robota (Log Message trafia tutaj)	Debugowanie
Immediate	Konsola do testowania wyrazen VB.NET "na zywo"	Testowanie wyrazen bez uruchamiania robota
Project	Drzewo plikow projektu	Nawigacja po projekcie

Tryby projektowania — Modern vs Classic

UiPath Studio oferuje dwa tryby projektowania aktywnosci UI:

- **Modern Design** (domyslny od wersji 2021.10+) — uzywa `Use Application/Browser`, `Click`, `Type Into` z nowym silnikiem selektorow (Unified Target). Zalecany dla nowych projektow.
- **Classic Design** — starszy tryb z `Open Browser`, `Attach Browser`, klasycznymi selektorami XML. Nadal spotykany w istniejacych projektach.

> **Na szkoleniu uzywamy Modern Design.**

Selektory — jak robot "widzi" elementy UI?

Selektor to "adres" elementu na ekranie. Gdy klikasz przycisk na stronie, UiPath zapisuje jego selektor — unikalny zestaw atrybutow, ktory pozwala go znalezc przy nastepnym uruchomieniu.

Typy selektorow w Modern Design

Typ	Opis	Kiedy uzywac
Strict Selector	Dokladny selektor XML z konkretnymi atrybutami (id, name, class)	Gdy element ma stabilne, unikalne atrybuty
Fuzzy Selector	Selektor oparty na "podobienstwie" — toleruje drobne zmiany	Gdy atrybuty moga sie lekko zmieniac
Image	Rozpoznawanie wizualne (zrzut ekranu elementu)	Ostatecznosc — gdy brak selektorow (np. Citrix, VDI)

Hierarchia niezawodnosci selektorow

```
Strict Selector (najlepszy) → Fuzzy Selector → Image (najgorszy)
```

Robot probuje w tej kolejnosci — jesli Strict nie zadziala, sprobuje Fuzzy, potem Image. Dlatego warto miec wszystkie trzy skonfigurowane.

Typowe atrybuty selektora

- `id` — najbardziej stabilny (np. `id='txtName'`)
- `name` — czesto stabilny
- `class` — zmienia sie przy aktualizacjach UI
- `idx` (indeks) — najgorszy — zmienia sie gdy dodasz/usuniesz element na stronie

> **Zasada:** Im bardziej "wewnetrzny" atrybut (id, name), tym stabilniejszy selektor. Unikaj `idx` i `tableRow` gdy to mozliwe.

Use Application/Browser — izolacja kontekstu

`Use Application/Browser` to kontener, ktory "podpina" robota do konkretnej aplikacji lub okna przegladarki. Wszystkie aktywnosci UI wewnatrz tego kontenera dzialaja w kontekscie tej aplikacji.

Dlaczego to wazne

- **Izolacja** — robot wie, w ktorym oknie przegladarki klikac (wazne gdy masz wiele otwartych kart)
- **Zarzadzanie cyklem zycia** — mozna skonfigurowac: otworz aplikacje na poczatku, zamknij na koncu
- **CloseMode** — `Always` (zamknij po zakonczeniu), `Never` (zostaw otwarta), `OnError` (zamknij tylko przy bledzie)

Modul 2 — Czym jest zmienna?

Zmienna to "pudelko z etykieta". Na etykiecie jest **nazwa** (np. `str_imie`), pudelko ma okreslony **rozmiar i ksztalt** (typ danych — String, Integer, etc.), a w srodku jest **zawartosc** (wartosc).

Kluczowe cechy zmiennej

- **Nazwa** — jak ja nazywamy (np. `str_imie`, `int_wiek`)
- **Typ danych** — co moze przechowywac (tekst, liczbe, date...)
- **Scope** — gdzie jest widoczna (w ktorym "pokoju" moze byc uzywana)
- **Wartosc domyslna** — co jest w pudelku na poczatku (opcjonalnie)

Konwencja nazewnicza (zalecana)

Konwencja nazewnicza to umowa w zespole: "jak nazywamy zmienne, zeby kazdy wiedzial, co w nich jest, nawet bez czytania kodu". W UiPath najczesciej stosuje sie **prefiksy wegierskie** — trzy litery na poczatku nazwy wskazuja typ:

Prefiks	Typ	Opis	Przyklad wartosci
str_	String	Tekst	"Jan Kowalski"
int_	Integer (Int32)	Liczba calkowita	42
dbl_	Double	Liczba zmiennoprzecinkowa	3.14
dt_	DataTable	Tabela danych	(obiekt tabelaryczny)
dat_	DateTime	Data i czas	2024-01-15 10:30:00
arr_	Array	Tablica (staly rozmiar)	{"a", "b", "c"}
lst_	List	Lista (dynamiczny rozmiar)	Lista elementow
bln_ lub bl_	Boolean	Wartosc logiczna	True / False
dict_	Dictionary	Slownik klucz-wartosc	Pary klucz-wartosc
row_	DataRow	Wiersz tabeli	Jeden wiersz DataTable

> **Dlaczego to wazne?** Wyobraz sobie workflow z 30 zmiennymi. Bez konwencji widzisz: `wynik`, `dane`, `x`, `temp`. Z konwencja widzisz: `str_wynik_operacji`, `dt_dane_klientow`, `int_licznik`, `dat_data_raportu` — od razu wiesz, co to jest i jakiego typu.

Scope — zakres widocznosci zmiennej

Scope to "pokoj", w ktorym zmienna jest widoczna. Zmienna zadeklarowana w Sequence jest widoczna **tylko w tej Sequence** i jej "dzieciach" (zagniezdonych kontenerach).

```
Main Sequence ← zmienna "A" (widoczna wszedzie ponizej)
  ├── Sequence 1 ← zmienna "B" (widoczna tylko w Seq 1)
  │     ├── Activity 1 ← widzi: A, B
  │     └── Activity 2 ← widzi: A, B
  └── Sequence 2 ← zmienna "C" (widoczna tylko w Seq 2)
        ├── Activity 3 ← widzi: A, C (NIE widzi B!)
        └── Activity 4 ← widzi: A, C
```

Zasady Scope

1. Zmienna jest widoczna w Sequence, w ktorej jest zadeklarowana, i we **wszystkich zagniezdonych** kontenerach
2. Zmienna **NIE jest widoczna** w kontenerach "obok" (sibling) ani "wyzej" (parent)
3. Im wyzej w hierarchii zadeklarujesz zmienna, tym szersza widocznosc — ale nie przesadzaj! Deklaruj zmienna tam, gdzie jest naprawde potrzebna

> **Typowy blad:** Zmienna zadeklarowana wewnatrz petli `For Each` — po wyjsciu z petli zmienna nie jest dostepna. Rozwiazanie: zadeklaruj ja na poziomie Sequence zawierajacej petle.

Default Value — wartosc domyslna

Typ	Default gdy nie podasz	Przyklad jawnego default
String	`Nothing` (nie `""` !)	`""` (pusty string)
Int32	`0`	`1`
Double	`0.0`	`0.0`
Boolean	`False`	`True`
DataTable	`Nothing`	`New System.Data.DataTable`
DateTime	`01/01/0001 00:00:00`	`Now`

Wazne: `Nothing` to nie to samo co `""` (pusty string)! `Nothing` oznacza "brak obiektu" — proba wywolania metody na `Nothing` rzuci `NullReferenceException`. Dlatego zmienne String warto inicjalizowac jako `""`.

Typy danych — szczegoly

Typ	Rozmiar	Zakres / Opis	Typowe uzycie w RPA
`String`	Dynamiczny	Dowolny tekst (Unicode)	Imiona, adresy, tresci emaili, selektory
`Int32`	32 bity	-2,147,483,648 do 2,147,483,647	Liczniki, indeksy, ilosci
`Int64` (Long)	64 bity	Ogromny zakres	NIP, REGON, PESEL (za dlugie na Int32!)
`Double`	64 bity	Liczby zmiennoprzecinkowe	Kwoty, kursy walut, procenty
`Boolean`	1 bit	`True` / `False`	Flagi, warunki, wyniki sprawdzen
`DateTime`	—	Data i czas	Daty raportow, terminy, timestamps
`Object`	Dynamiczny	Dowolny typ	Gdy nie znamy typu z gory (np. dane z Excela)

Konwersja typow — kiedy i dlaczego

UiPath (VB.NET) jest typowany statycznie — nie mozesz dodac `String` do `Int32` bez konwersji. Typowe konwersje:

```
' String → Integer (gdy Input Dialog zwraca String z liczba)
cint("42")                    → 42

' String → Double (gdy potrzebujesz liczby zmiennoprzecinkowej)
cdbl("3.14")                  → 3.14

' Integer → String (gdy chcesz wyswietlic liczbe w tekscie)
42.ToString                   → "42"

' Integer → Double (gdy dzielisz dwie liczby calkowite)
cdbl(7) / cdbl(2)             → 3.5
' BEZ konwersji: 7 / 2 = 3 (obcina czesc dziesietna!)

' String → DateTime
DateTime.ParseExact("15-01-2024", "dd-MM-yyyy", Nothing)
```

Typowa pulapka — dzielenie Integer:

```
int_wynik = 7 / 2    → 3  (nie 3.5! Integer obcina!)
dbl_wynik = cdbl(7) / cdbl(2)  → 3.5  (poprawnie!)
```

Cwiczenia — Modul 1 i 2

Zadanie 1 — "Find Unicorn Name"

Input Dialog, Use Application/Browser, Type Into, Select Item, Click, Get Text

CEL

Automatyzacja strony www — pobranie danych od uzytkownika przez Input Dialog, wpisanie ich na stronie internetowej, odczytanie wyniku.

CZEGO SIE NAUCZYSZ

- Korzystanie z `Input Dialog` (w tym lista wyboru z `OptionsString`)
- Praca z `Use Application/Browser` — izolacja kontekstu przegladarki
- Aktywnosci UI: `Type Into`, `Select Item`, `Click`, `Get Text`
- Roznica miedzy Strict a Fuzzy Selector na zywym przykladzie

Zadanie 2 — Kalkulator

Dynamiczne selektory, string.Format, automatyzacja aplikacji desktopowej

CEL

Automatyzacja aplikacji desktopowej (Windows Calculator) z uzyciem dynamicznych selektorow.

CZEGO SIE NAUCZYSZ

Dynamiczne selektory — klucz do skalowalnosci robotow. Zamiast tworzyc osobny Click dla kazdej cyfry (0-9), selektor parametryzuje `automationid` wartoscia zmiennej

`string.Format` — czytelny sposob na wstawianie zmiennych do selektorow

Automatyzacja aplikacji desktopowych vs przegladarkowych

Dynamiczne selektory — jak to dziala

Bez dynamicznego selektora musialbys stworzyc 10 osobnych aktywnosci Click — po jednej dla kazdej cyfry. Z dynamicznym selektorem wystarczy **jeden Click**, a wartosc cyfry pochodzi ze zmiennej.

```
[string.Format("<uia automationid='num{0}Button' ... />", str_liczba)]
```

Dwa sposoby tworzenia dynamicznych selektorow

Sposob	Przyklad	Kiedy uzywac
`string.Format`	`string.Format("num{0}Button", str_liczba)`	Gdy masz jeden lub kilka parametrow
Konkatenacja	`"num" + str_liczba + "Button"`	Proste przypadki

Wskazowka: `string.Format` jest czytelniejszy gdy masz wiele parametrow: `string.Format("{0}_{1}_{2}", a, b, c)` vs `a + "_" + b + "_" + c`

Zadanie 3 — Dni tygodnia

Read Text File, Split(), For Each, IF

CEL

Praca z plikiem tekstowym, tablica (Array), petla For Each i warunkiem IF.

CZEGO SIE NAUCZYSZ

- `Read Text File` — odczyt pliku tekstowego do zmiennej
- `Split()` — dzielenie tekstu na tablice po separatorze
- `For Each` — iteracja po elementach tablicy
- `IF` — logika warunkowa

Split() — jak dziala

`Split()` to metoda typu String, ktora dzieli tekst na tablice (Array) na podstawie separatora:

```
"A;B;C".Split(";")          → {"A", "B", "C"}     ' tablica 3-elementowa
"Jan Kowalski".Split(" ")    → {"Jan", "Kowalski"}  ' podzial po spacji
"1,2,3,4".Split(",")        → {"1", "2", "3", "4"} ' dane CSV
```

Przydatne metody String

Metoda	Opis	Przyklad
`.Split(separator)`	Dzieli tekst na tablice	`"A;B".Split(";")` → `{"A", "B"}`
`.Contains("X")`	Czy tekst zawiera X?	`"Hello".Contains("ell")` → `True`
`.Replace("X","Y")`	Zamien X na Y	`"ABC".Replace("B","X")` → `"AXC"`
`.Trim()`	Usun spacje z poczatku i konca	`" text ".Trim()` → `"text"`
`.ToUpper()` / `.ToLower()`	Wielkie/male litery	`"abc".ToUpper()` → `"ABC"`
`.Length`	Dlugosc tekstu	`"Hello".Length` → `5`
`.Substring(start, len)`	Wytnij fragment	`"Hello".Substring(0,3)` → `"Hel"`

Array vs List — fundamentalna roznica

Cecha	Array (Tablica)	List (Lista)
Rozmiar	**Staly** — ustalony przy tworzeniu	**Dynamiczny** — rosnie w miare potrzeb
Dodawanie	Niemozliwe (trzeba stworzyc nowa, wieksza tablice)	`Add To Collection`
Usuwanie	Niemozliwe	`Remove From Collection`
Tworzenie	`{"A", "B", "C"}`	`New List(Of String)`
Kiedy uzywac	Gdy znasz liczbe elementow z gory (np. dni tygodnia)	Gdy nie znasz z gory (np. emaile, pliki)

Zadanie 4 — Flowchart Kalkulator

Flowchart, FlowSwitch, FlowDecision, konwersja typow

CEL

Nauka Flowchart, FlowSwitch i FlowDecision — alternatywnych kontenerow do budowania logiki.

CZEGO SIE NAUCZYSZ

- Flowchart jako alternatywa dla Sequence — kiedy jest czytelniejszy
- `FlowSwitch` — wielogaleziowe rozgalezienie (zamiast wielu zagniezdonych IF/Else)
- `FlowDecision` — logika warunkowa w Flowchart
- Konwersja typow: `cint()` (String → Integer), `cdbl()` (Integer → Double)

Flowchart vs Sequence — kiedy co wybrac?

```
Sequence:                    Flowchart:

┌──────────────┐             ┌──────────┐
│ Input Dialog │             │  Start   │
│ Input Dialog │             └────┬─────┘
│ Input Dialog │                  ↓
│ IF dodawanie │             ┌──────────┐
│   ELSE IF... │             │  Dane    │
│   ELSE IF... │             └────┬─────┘
│   ELSE IF... │                  ↓
│ Message Box  │             ┌──────────┐
│ IF > 10      │             │FlowSwitch│
│   Message    │             ├──┬──┬──┬─┤
└──────────────┘            +  -  *  /
                            ↓  ↓  ↓  ↓
                         (wynik → Decision → Message)
```

Flowchart jest czytelniejszy gdy masz **wiele rozgalezien** (4 operacje + decyzja).

Typowa pulapka — dzielenie bez cdbl:

```
' ZLE: 7 / 2 = 3 (Integer dzielenie!)
dbl_wynik = int_liczba1 / int_liczba2

' DOBRE: cdbl(7) / cdbl(2) = 3.5
dbl_wynik = cdbl(int_liczba1) / cdbl(int_liczba2)
```

Modul 3 — Czym jest DataTable?

DataTable to typ danych w UiPath (i .NET), ktory przechowuje dane tabelaryczne — wiersze i kolumny, dokladnie jak arkusz Excela. Roznica: DataTable istnieje **w pamieci robota**, a nie w pliku na dysku.

Dlaczego to wazne

- **Szybkosc** — operacje na DataTable w pamieci sa wielokrotnie szybsze niz odczyt/zapis do pliku Excel przy kazdej operacji
- **Elastycznosc** — mozesz filtrowac, sortowac, dodawac kolumny bez otwierania Excela
- **Wzorzec pracy:** Wczytaj Excel → przetwarz w pamieci (DataTable) → zapisz wynik do nowego Excela

```
[Plik Excel] --Read Range--> [DataTable w pamieci] --operacje--> [DataTable] --Write Range--> [Nowy plik Excel]
```

Kluczowe aktywnosci DataTable

Aktywnosc	Opis	Wazne szczegoly
`Build Data Table`	Definicja struktury (kolumny + typy) + dane startowe	Edytor wizualny — kliknij "DataTable..."
`Add Data Row`	Dodaje wiersz przez `ArrayRow` = `{val1, val2, val3}`	Liczba wartosci MUSI odpowiadac liczbie kolumn!
`Add Data Column`	Dodaje nowa kolumne do istniejacej tabeli	Typ domyslny: Object (przyjmie wszystko)
`Remove Data Column`	Usuwa kolumne po nazwie lub indeksie	Ostroznie z indeksami — po usunieciu sie przesuwaja!
`For Each Row`	Iteracja — `row.Item(0)` lub `row.Item("NazwaKolumny")`	Typ iteratora: `DataRow`
`Filter Data Table`	Filtrowanie wierszy i/lub kolumn	Dwa tryby: Keep (zostaw pasujace) / Remove (usun pasujace)
`Sort Data Table`	Sortowanie wedlug kolumny (Ascending/Descending)	Tworzy NOWY DataTable (oryginal niezmieniony)
`Output Data Table`	Konwersja tabeli do String (format CSV)	Przydatne do debugowania — loguj tabele jako tekst
`Write Range`	Zapis DataTable do pliku Excel	AddHeaders = True (domyslnie) — zapisuje nazwy kolumn

Dostep do danych w DataTable

```
' W petli For Each Row:
row.Item(0)                    ' pierwszy element (indeks od 0)
row.Item("Imie")               ' element po nazwie kolumny (bezpieczniejsze!)
row.Item(0).ToString           ' konwersja do tekstu
cint(row.Item(2).ToString)     ' konwersja do Integer

' Bezposredni dostep (bez petli):
dt_tabela.Rows(0).Item("Imie")  ' pierwszy wiersz, kolumna "Imie"
dt_tabela.Rows.Count           ' liczba wierszy
dt_tabela.Columns.Count        ' liczba kolumn
```

Po indeksie vs po nazwie — co lepsze?

Dostep	Przyklad	Zaleta	Wada
Po indeksie	`row.Item(0)`	Krotszy zapis	Zmieni sie gdy dodasz/usuniesz kolumne!
Po nazwie	`row.Item("Imie")`	Odporny na zmiany struktury	Dluzszy zapis, literowka = wyjatek

> **Rekomendacja:** W produkcji zawsze uzywaj dostepu **po nazwie** — jest bezpieczniejszy.

Typowe pulapki przy pracy z DataTable

1. Typ Object

Dane wczytane z Excela przez Read Range sa typu `Object`, nie `String` czy `Integer`. Dlatego zawsze potrzebujesz `.ToString` lub `cint()`:

```
' ZLE — Object nie ma metody Length:
row.Item("Imie").Length

' DOBRE — najpierw konwertuj:
row.Item("Imie").ToString.Length
```

2. Indeksowanie od 0

Pierwszy wiersz to `Rows(0)`, pierwsza kolumna to `Item(0)`:

```
dt.Rows(0).Item(0)   ' pierwszy wiersz, pierwsza kolumna
dt.Rows(1).Item(2)   ' drugi wiersz, trzecia kolumna
```

3. AddHeaders w Read Range

Jesli `AddHeaders = True`, pierwszy wiersz Excela staje sie **nazwami kolumn**, nie danymi. Jesli `False`, kolumny maja nazwy Column1, Column2...

Filter Data Table — szczegoly konfiguracji

Filter Data Table ma dwa panele:

1. Filter Rows — filtrowanie wierszy (warunki na wartosci)

- **Column:** nazwa lub indeks kolumny
- **Operation:** `=`, `<>`, `>`, `<`, `>=`, `<=`, `Contains`, `StartsWith`, `EndsWith`
- **Value:** wartosc do porownania
- **Tryb:** `Keep` (zostaw pasujace) lub `Remove` (usun pasujace)

2. Filter Columns — filtrowanie kolumn (ktore kolumny zachowac/usunac)

- **Tryb:** `Keep` (zostaw wymienione) lub `Remove` (usun wymienione)

> **Wazne:** Oba panele dzialaja **jednoczesnie** — mozesz filtrowac wiersze AND usuwac kolumny w jednym kroku.

Cwiczenia — Modul 3

Cwiczenie: DataTable — Wprowadzenie

Build Data Table, Add Data Row, For Each Row, Filter, Sort, Write Range

CEL

Pelny workflow na DataTable — tworzenie tabeli, dodawanie wierszy i kolumn, iteracja, filtrowanie, sortowanie, zapis do Excela.

CZEGO SIE NAUCZYSZ

- `Build Data Table` — tworzenie tabeli z edytorem wizualnym
- `Add Data Row` / `Add Data Column` — rozbudowa tabeli w runtime
- `For Each Row` — iteracja po wierszach z modyfikacja danych
- `Filter Data Table` / `Sort Data Table` — przetwarzanie danych
- `Output Data Table` — konwersja tabeli do tekstu (debugowanie)
- `Write Range` — zapis do Excela

Cwiczenie: Excel — Wprowadzenie

Read Range Workbook, Now, ToString formatowanie, dynamiczne nazwy plikow

CEL

Read Range + dynamiczna nazwa pliku wynikowego z data.

CZEGO SIE NAUCZYSZ

- `Read Range Workbook` — wczytanie pliku Excel do DataTable
- `Now` — pobranie biezacej daty i czasu
- `.ToString("dd-MM-yyyy")` — formatowanie daty
- Dynamiczne nazwy plikow — klucz w produkcji (unikalnosc raportow)

Formatowanie daty — przydatne wzorce

Wzorzec	Przyklad	Kiedy uzywac
`"dd-MM-yyyy"`	15-01-2024	Nazwy plikow (polski format)
`"yyyy-MM-dd"`	2024-01-15	Sortowalne nazwy plikow (ISO)
`"dd.MM.yyyy HH:mm"`	15.01.2024 14:30	Logi, raporty
`"MMMM yyyy"`	January 2024	Naglowki raportow
`"yyyyMMdd_HHmmss"`	20240115_143022	Unikalne nazwy plikow

> **Wskazowka:** Format `"yyyy-MM-dd"` jest najlepszy do nazw plikow, bo pliki sortuja sie chronologicznie w eksploratorze.

Read Range vs Read Range Workbook

Cecha	Read Range	Read Range Workbook
Wymaga zainstalowanego Excela	TAK	NIE
Otwiera Excel w tle	TAK	NIE (czyta plik bezposrednio)
Szybkosc	Wolniejszy	Szybszy
Obsluguje formuly	TAK (odczytuje wynik)	Ograniczona
Zalecany	Gdy potrzebujesz formul	Domyslny wybor

Zadanie 1 Excel — Produkty

Filter Data Table, Add Data Column, For Each Row, obliczenia, Write Range

CEL

Filtrowanie danych, usuwanie kolumny, obliczenia w petli i zapis raportu do Excela.

CZEGO SIE NAUCZYSZ

- `Filter Data Table` — filtrowanie wierszy AND usuwanie kolumn jednoczesnie
- `Add Data Column` — dodawanie kolumny obliczeniowej w runtime
- Obliczenia w petli `For Each Row` — `cint()` niezbedne gdy dane z Excela sa typem `Object`

Zadanie 2 Excel — Kursy walut NBP

Extract Table Data, Split().Last, Filter Data Table, dynamiczny zapis

CEL

Data Scraping ze strony internetowej, filtrowanie DataTable, dynamiczny zapis do pliku.

CZEGO SIE NAUCZYSZ

- `Extract Table Data` — automatyczne wyodrebnianie danych tabelarycznych ze stron www
- `.Split(" ").Last` — elegancki sposob na wyciagniecie ostatniego slowa z tekstu
- `Filter Data Table` — wyodrebnienie jednego wiersza z tabeli
- `Rows(0).Item(2)` — dostep do konkretnej komorki po indeksach

Extract Table Data — jak to dziala

`Extract Table Data` (dawniej Data Scraping) to narzedzie do automatycznego wyodrebniania danych tabelarycznych ze stron www:

1. Wskaz pierwszy element tabeli (np. pierwsza komorke)
2. UiPath automatycznie rozpozna strukture tabeli
3. Wskaz drugi element (w tym samym wierszu lub kolumnie) — UiPath potwierdzi wzorzec
4. Konfiguruj kolumny (nazwy, typy, URL-e)
5. Wynik: DataTable z wszystkimi danymi

Przydatne metody String

```
' .Split(" ").Last — wyciagnij ostatnie slowo:
"Tabela kursow z dnia 2024-01-15".Split(" ").Last  → "2024-01-15"

' .Split(" ").First — wyciagnij pierwsze slowo:
"Jan Kowalski".Split(" ").First  → "Jan"

' .Rows(0).Item(2) — konkretna komorka DataTable:
dt_euro.Rows(0).Item(2).ToString  → "4,5545"
```

Przydatne skroty klawiszowe

Skrot	Akcja
Ctrl+K	Szybkie tworzenie zmiennej (w polu aktywnosci)
Ctrl+M	Szybkie tworzenie argumentu
F2	Pauza 3 sekundy przy wskazywaniu elementow (przydatne dla hover)
F5	Uruchom w trybie Debug
F11	Step Into — wykonaj jedna aktywnosc
F10	Step Over — wykonaj aktywnosc i jej "dzieci"

Typowe problemy i rozwiazania

Problem	Przyczyna	Rozwiazanie
Zmienna poza Scope	Element nie widzi zmiennej	Sprawdz gdzie jest zadeklarowana — przesun wyzej
Selektor nie dziala	Strona sie zmienila lub element dynamiczny	Uzyj Fuzzy Selector lub UI Explorer
`cint` vs `cdbl`	Blad konwersji typow	Zawsze konwertuj przed operacja matematyczna
`NullReferenceException`	Zmienna nie zainicjalizowana	Ustaw Default Value (np. `""` dla String)
Input Dialog zwraca String	Wszystko z Input Dialog jest String!	Konwertuj: `cint(str_wynik)`
Rozszerzenie przegladarki	Brak wtyczki Chrome/Edge	Home → Tools → UiPath Extensions